

POLITECNICO
MILANO 1863

Bioinformatics Algorithms

Read mapping (part 3)

Rosario M. Piro
(based on slides by Peter N. Robinson)

Dipartimento di Elettronica, Informazione e Bioingegneria (DEIB)
Politecnico di Milano

BWT/FM Index algorithms for read mapping

There are lots of published read aligners for genomic resequencing. Perhaps the best known amongst them use the BWT/FM Index plus lots of **bells and whistles**

- **BWA**: Li H, Durbin R (2009). Fast and accurate short read alignment with Burrows-Wheeler transform. *Bioinformatics* **25**:1754-60
- **BWA-MEM**: Li H (2013). Aligning sequence reads, clone sequences and assembly contigs with BWA-MEM. *arXiv*:1303.3997v2
- **Bowtie**: Langmead B, Trapnell C, Pop M, Salzberg SL (2009). Ultrafast and memory-efficient alignment of short DNA sequences to the human genome. *Genome Biol* **10**:R25
- **SOAP2**: Li R et al (2009). SOAP2: an improved ultrafast tool for short read alignment. *Bioinformatics*. **25**:1966-7

- More short-read sequence aligners at: `https://en.wikipedia.org/wiki/List_of_sequence_alignment_software#Short-read_sequence_alignment`
  - Many of these are based on the BWT!

Here, we will concentrate on BWA ...

Burrows Wheeler Aligner – BWA

The nomenclature and descriptions used in the BWA paper are different in a few ways to those used in this lecture

- Only some aspects of the paper can be presented here
- **Exact matching** is performed roughly as we have seen before
  - Based on the BWT (of course ...)
  - We will see this now in a more fomalized way
- A major issue that needs to be solved by any practical read mapper is **inexact matching**
  - Individual reads may differ from the reference genome, e.g., due to ...
    - ★ sequencing errors
    - ★ single nucleotide variants ($\sim$1 out of 1000 bases differ)
- We will introduce the topic of inexact matching with the brute force approach that is mentioned (and rejected) in the introduction of the BWA paper

Inexact matching: problem definition

Given:

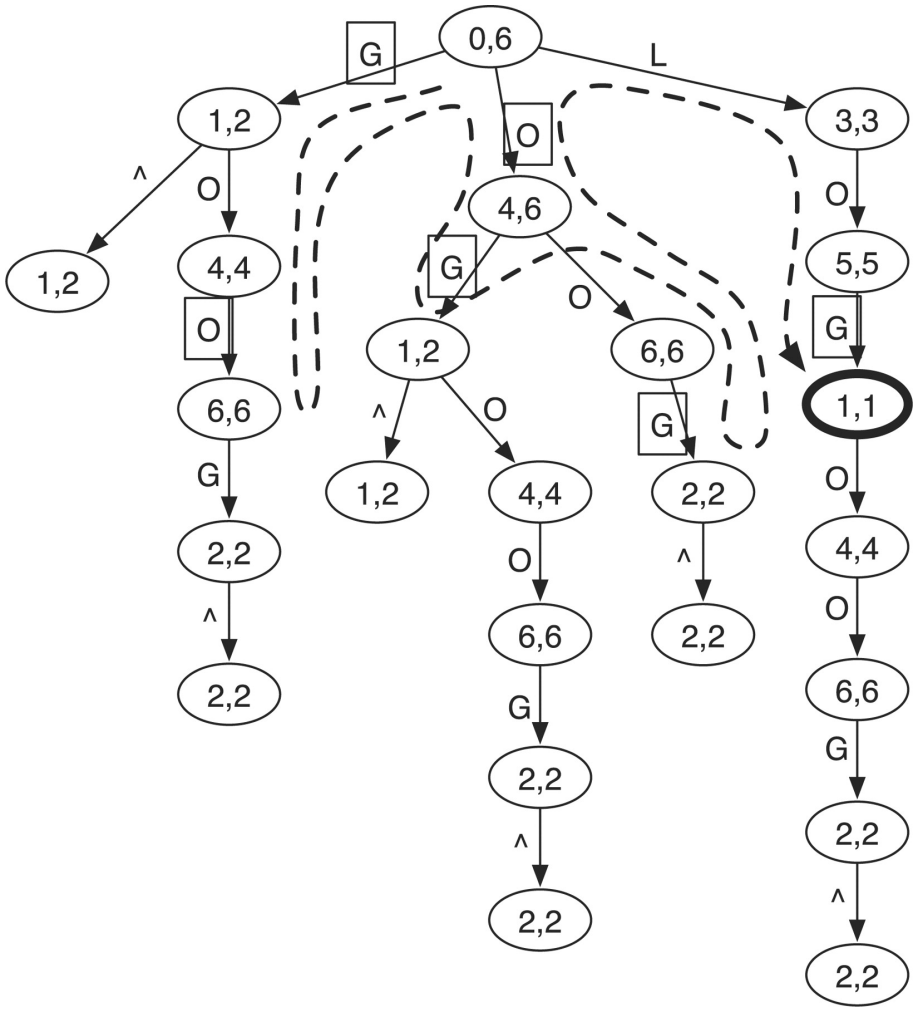
- Let W (search word/pattern) and T (input "text"/string, e.g., reference genome) be two strings over the same alphabet Σ, e.g., $\Sigma = \{\text{A,C,G,T}\}$
- Let $|W| = m$ and $|T| = n$
- Let $d(a, b)$ be a distance measure for two strings a and b (e.g., Hamming distance or edit distance)
- Let d_{max} be a threshold for the allowed maximum distance
- Let $T^p = T[p \ldots p + (m-1)]$ be the substring of T having length m (length of search pattern) starting at position p
  - We assume the substring in T to be of the same length as the pattern W, but this is actually not necessarily the case if we use the edit distance and allow for gaps

Find:

- Does at least one T^p exist such that $d(W, T^p) \leq d_{max}$? (yes/no)
- If so, how many p with $0 \leq p \leq n - m$ exist such that $d(W, T^p) \leq d_{max}$?
- List all p for which $d(W, T^p) \leq d_{max}$ (i.e., report all matching positions for W in T with at most d_{max} errors)
- Alternatively: find the best match regardless of distance and report a "mapping quality" (somehow score the quality of the match)

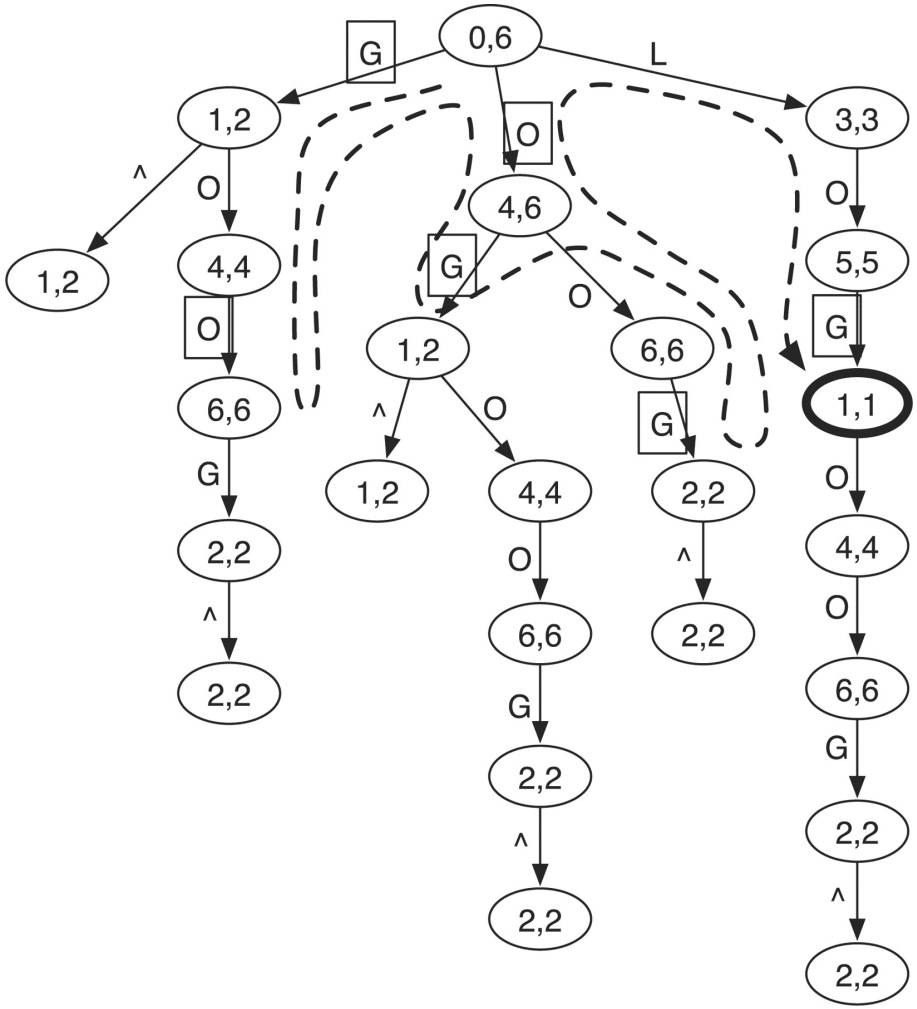
Prefix trie

- Prefix trie for string T: equivalent to suffix trie, but a *path from a **leaf** to the **root*** gives a unique prefix of T
  - Thus, *path from **root** to **leave*** gives the reverse of a prefix!
  - The prefix trie of T is identical to the suffix trie of the reverse of T!
  - Example shown here: T="^GOOGOL" (note the use of a *start* character!)
  - The *reverse* of a substring of T is a path from root to some node
    - ★ Example: the reverse of substring "GOL" is the path from the root to the **bold** node ($\rightarrow$"L"$\rightarrow$"O"$\rightarrow$"G")



Prefix trie

- Like for the BWT-based approach, on a prefix trie we can search for substrings starting with the last character of the substring!
  - Example: to search for "GOL", we start with "L"!
- Depth-first search (DFS) to search for a substring!
- Inexact matching (brute-force):
  - DFS but allow for up to d_{max} mismatches!
  - Example: search word "**LOL**" with $d_{max} = 1$: DFS path is dashed line! Letters with mismatches are shown in boxes, for example, "[G]" ...



Note: the numbers associated with nodes are suffix array intervals

Suffix array interval

BWA uses the following notation for a **"suffix array interval"**

- All occurrences of a substring W in the original string T appear next to each other in the suffix array (SA), because a substring is equivalent to the prefix of a suffix of T and we have lexicographically sorted all suffixes!
  $\rightarrow$ A substring W can be represented as an SA interval:

$$\left[\underline{R}(W), \overline{R}(W)\right]$$

- For instance, in the example shown to the right (T=googol\$), the SA interval of substring "go" is $[1,2]$ and the suffix array interval of "o" is $[4,6]$
  - Important: the indices $\underline{R}(W)$ and $\overline{R}(W)$ do not directly refer to positions in the original text T, but instead define a section of the suffix array (shown in dashed lines) which in turn contains references to the original text

```
T=googol$
 0123456

0  6  $googo l
1  3  gol$go o
2  0  googol $
3  5  l$goog o
4  2  ogol$g o
5  4  ol$goo g
6  1  oogol$ g
```

(i) (SA) (BWM)

BWA: perfect matching

The BWA paper presents the method of calculating the SA interval of the query/search word $W = W[0 \ldots m-1]$ that we have already seen, using a slightly different notation:

- Can be done iteratively, starting from the end of W:

$$\begin{aligned} \underline{R}(W_k) &= \underline{R}(a_k W_{k+1}) \\ &= C(a_k) + Occ(a_k, \underline{R}(W_{k+1}) - 1) + 1 \\ \overline{R}(W_k) &= \overline{R}(a_k W_{k+1}) \\ &= C(a_k) + Occ(a_k, \overline{R}(W_{k+1})) \end{aligned}$$

0	6	$googo l
1	3	gol$go o
2	0	googol $
3	5	l$goog o
4	2	ogol$g o
5	4	ol$goo g
6	1	oogol$ g

where

- $W_k = W[k \ldots m-1]$; suffix of W with $0 \leq k \leq m-1$
- a_k is the k-th character of W: $a_k = W[k,k]$
  - Thus $W_k = a_k W_{k+1}$
- $C(a_k)$ = number of chars in $T[0, n-2]$ (i.e., in T without the final \$) that are lexicographically smaller than a_k
- $Occ(a_k, i)$ = number of occurrences of a_k in $BWT[0, i]$

- Initialization: $W_m = ""$; $\underline{R}("") = 0$ and $\overline{R}("") = |T| - 1$
  (We haven't searched for a letter yet; we begin with the entire SA)

Example for perfect matching

Example for perfect matching: find the SA interval for query W="go" in T="googol$"

- Step 1: search for "o" (last character)

$$
\begin{aligned}
\underline{R}(\text{"}o\text{"}) &= C(\text{"}o\text{"}) + Occ(\text{"}o\text{"}, \underline{R}(\text{""}) - 1) + 1 \\
&= 3 + 0 + 1 = 4 \\
\overline{R}(\text{"}o\text{"}) &= C(\text{"}o\text{"}) + Occ(\text{"}o\text{"}, \overline{R}(\text{""})) \\
&= 3 + 3 = 6
\end{aligned}
$$

0	6	$googo l
1	3	gol$go o
2	0	googol $
3	5	l$goog o
4	2	ogol$g o
5	4	ol$goo g
6	1	oogol$ g

- Step 2: search for "go" (last two characters)

$$
\begin{aligned}
\underline{R}(\text{"}go\text{"}) &= C(\text{"}g\text{"}) + Occ(\text{"}g\text{"}, \underline{R}(\text{"}o\text{"}) - 1) + 1 \\
&= 0 + 0 + 1 = 1 \\
\overline{R}(\text{"}go\text{"}) &= C(\text{"}g\text{"}) + Occ(\text{"}g\text{"}, \overline{R}(\text{"}o\text{"})) \\
&= 0 + 2 = 2
\end{aligned}
$$

BWA: precalculations

- We can calculate $C(a)$ for T='googol\$', defined in the paper as the number of symbols in $T[0, n-2]$ that are lexicographically smaller than $a \in \Sigma$
- Let us assume $\Sigma = \{g, l, o\}$
- The vector **C** is then:[1]

a	$C(a)$
g	0
l	2
o	3

- We also calculate $Occ(a, i)$, the number of occurrences of a in $BWT[0, i]$:[2]

i	a	$Occ(''g'', i)$	$Occ(''l'', i)$	$Occ(''o'', i)$
0	l	0	1	0
1	o	0	1	1
2	\$	0	1	1
3	o	0	1	2
4	o	0	1	3
5	g	1	1	3
6	g	2	1	3

[1] Equivalent to **F**, but cumulative (cumulative index property!)

[2] Equivalent to the tally table to determine the B-rank

Inexact matching

The overall algorithm looks like this

Algorithm 1 InexactSearch(W, z)

1: CalculateD(W)
2: **return** `InexRecur`$(W, |W|-1, z, 1, |T|-1)$

- `CalculateD`(W) precalculates lower bounds of the number of mismatches
- `InexRecur`(W, i, z, k, ℓ) returns the SA intervals of substrings in T that match W with no more than z differences
  - *Recursive* function:
  - For query word/pattern W ...
  - search for matches to $W[0..i]$ (here: we start with the full word, so $i = |W| - 1$) ...
  - with a max number of z mismatches (here: start with $z = d_{max}$) ...
  - on the condition that the suffix W_{i+1} already matches the SA interval $[k..\ell]$ (here: start with W_{i+1}="" and thus $k = 1$ and $\ell = |T| - 1$, i.e., full interval)

Let's see more details ...

Inexact matching

Let us examine the `CalculateD(W)` algorithm

Algorithm 2 CalculateD(W)

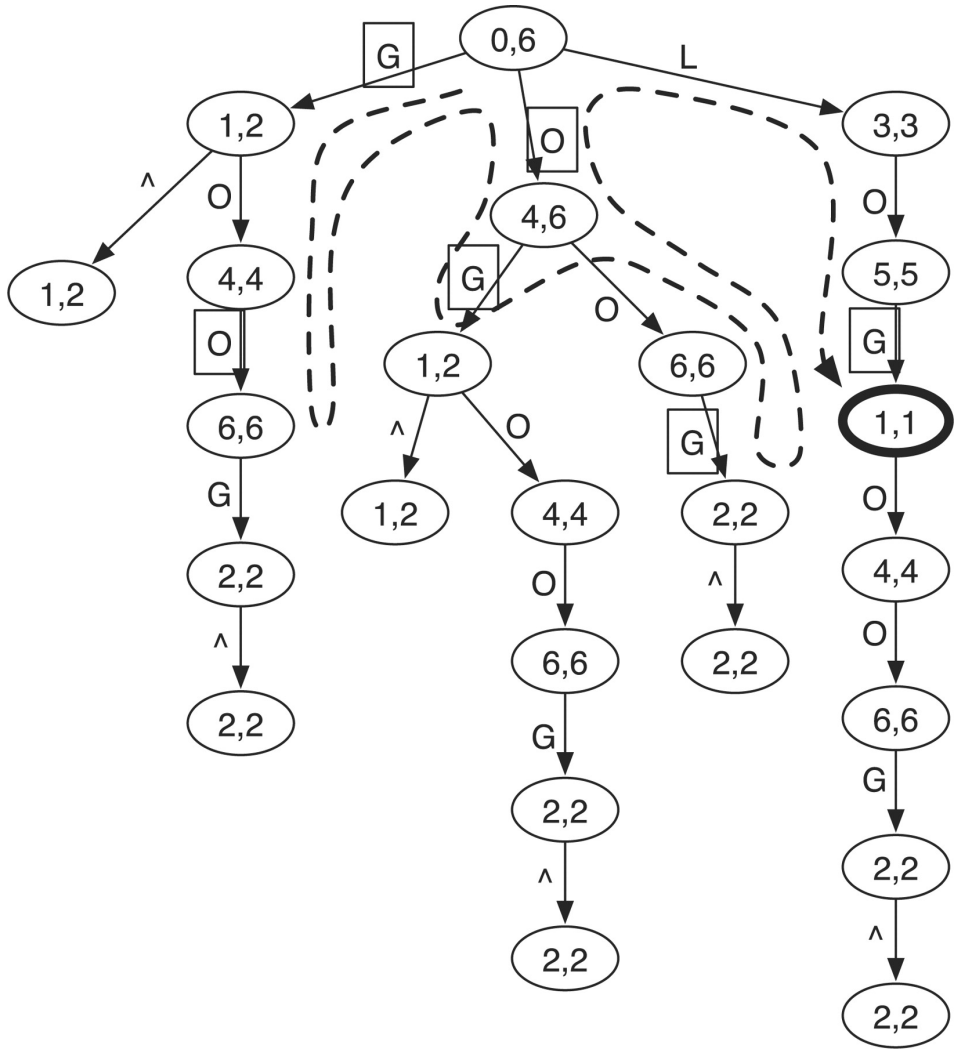
```
1:  z ← 0
2:  j ← 0
3:  for i = 0 to |W| − 1 do
4:      if W[j...i] is not a substring of T then
5:          z ← z + 1
6:          j ← i + 1
7:      end if
8:      D(i) ← z
9:  end for
10: return D
```

- $D(i)$ = **lower bound** of the number of differences in $W[0..i]$ to the best match in T
- Example: start with $z = i = j = 0$; check $W[0...0]$
  - If $W[0...0]$ is in T: $D(0) = 0$; increase i and check $W[0...1]$
  - If $W[0...0]$ is not in T: increase z; set $j = i + 1$ (after current substring!); $D(0) = 1$; increase i and check $W[1...1]$

Inexact matching

Retionale for `CalculateD(W)`: $\rightarrow D(1) = 1$

- Assume we use a *depth-first search* for inexact matches (as shown here)
  - (BWA uses *breadth-first search*)
- `CalculateD(W)` is a heuristic that allows to **stop** the DFS **early**:
  - Example: search for W="LOL" in T="^GOOGOL" with $d_{max} = 1$
  - First compare the *last letter* ("L") with the "G" on the left: 1st error
  - If at this point, we already know that for the rest of the search pattern *there will also be at least 1 mismatch*, then we know that the left branch of the trie will lead to at least 2 mismatches and we can avoid descending further!
  - How do we know? Because $W[0 \ldots 1]$="LO" is not in T!



Inexact matching

Algorithm 2 CalculateD(W)

```
1: z ← 0
2: j ← 0
3: for i = 0 to |W| − 1 do
4:     if W[j..i] is not a substring of T then
5:         z ← z + 1
6:         j ← i + 1
7:     end if
8:     D(i) ← z
9: end for
10: return D
```

- For T="GOOGOL$" and W="LOL", the `for` loop goes from 0 to 2
- We obtain:
  - $D(0) = 0$, because "L" (first letter) is in T
  - $D(1) = 1$, because "LO" (first two letters) is not in T
  - $D(2) = 1$, because "L" (next letter after "LO") is in T

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}      //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Lines 1-3

- If $i < 0$ then we are arriving from a recursive call where we have finished matching W (potentially including up to z mismatches)
- We return the SA interval $\{k, \ell\}$ representing the found hits

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}      //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Lines 4-6

- If the lower bound on the number of differences in $W[0 \ldots i]$ is already more than the maximum number of (still) allowed mismatches z, give up
- Return `null` (no match for $W[0 \ldots i]$ found)

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}     //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Line 7

- For this recursion, initialize the **current SA interval** I to the empty set

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}      //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Line 8

- Loop over all nucleotides (A,C,G,T), looking for a match...

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}      //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Lines 9-11

- Figure out the SA interval in **F** where the current nucleotide b would be
  - The SA interval is computed as for exact matching, just that we try all nucleotides!
- Check whether this interval is empty (no valid interval for $k > \ell$)

Inexact matching

Algorithm 3 InexRecur(W, i, z, k, ℓ)

```
1: if i < 0 then
2:     return {k, ℓ}      //i.e., an SA interval
3: end if
4: if z < D(i) then
5:     return ∅
6: end if
7: I ← ∅
8: for each b ∈ {a, c, g, t} do
9:     k ← C(b) + O(b, k − 1) + 1
10:    ℓ ← C(b) + O(b, ℓ)
11:    if k ≤ ℓ then
12:        if b = W[i] then
13:            I ← I ∪ InexRecur(W, i − 1, z, k, ℓ)     //match
14:        else
15:            I ← I ∪ InexRecur(W, i − 1, z − 1, k, ℓ)     //mismatch, decrement z
16:        end if
17:    end if
18: end for
19: return I
```

Lines 12-15

- If we have a match, then decrement i and keep going, thus checking $W[0 \ldots i-1]$
- If we have a mismatch, then decrement i and also z and keep going, thus allowing one less mismatch for $W[0 \ldots i-1]$!

Inexact matching

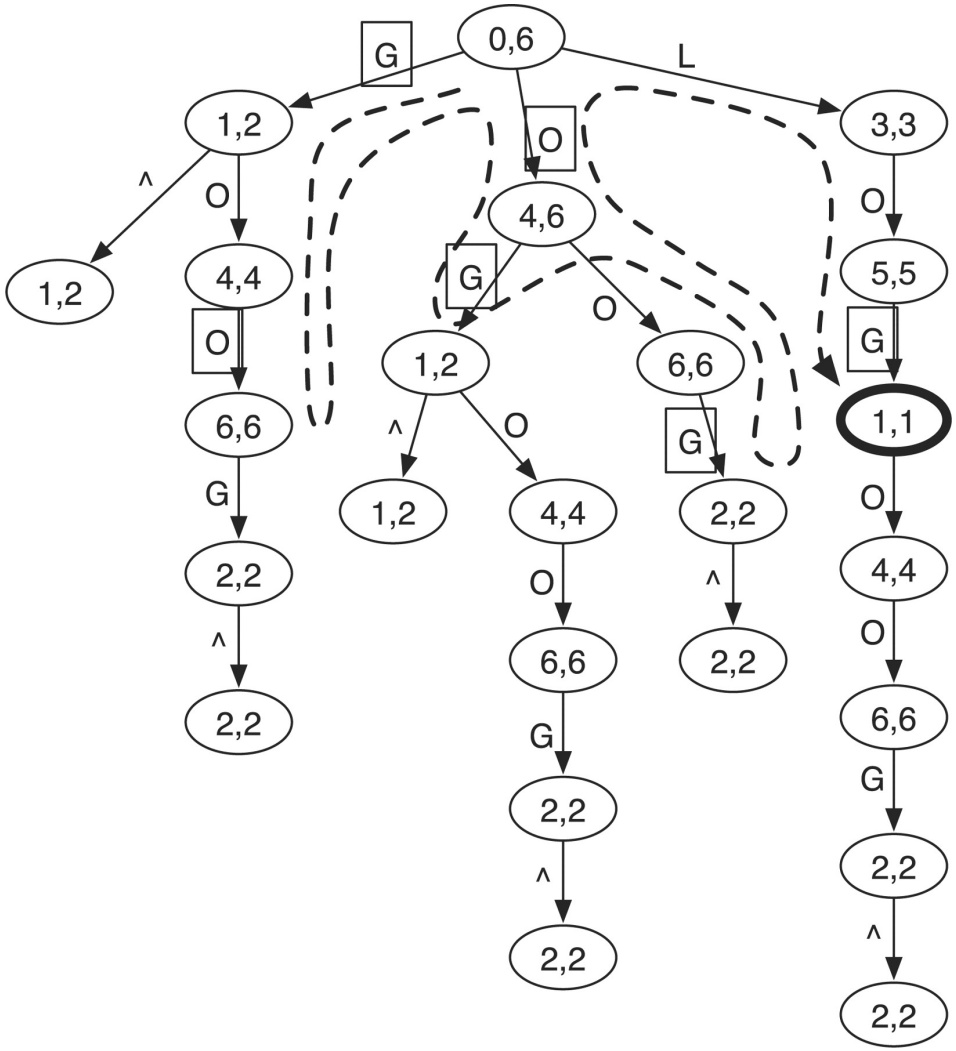
Consider now the role of the D matrix in the DFS shown in the figure

- The initial call to $\texttt{InexRecur}(W, i, z, k, \ell)$ (with W="LOL" and T="GOOGOL\$" and maximally one mismatch allowed) is:

  $\texttt{InexRecur}(W, |W|-1, z, 1, |T|-1)$
  i.e., $\texttt{InexRecur}(W, 2, 1, 1, 6)$

- The DFS skips lines 1-7 and from the root node chooses the character 'G'

- 'G' does not match the last character of 'LOL', so there is a mismatch, and we recursively call `InexRecur`, after decrementing i and z

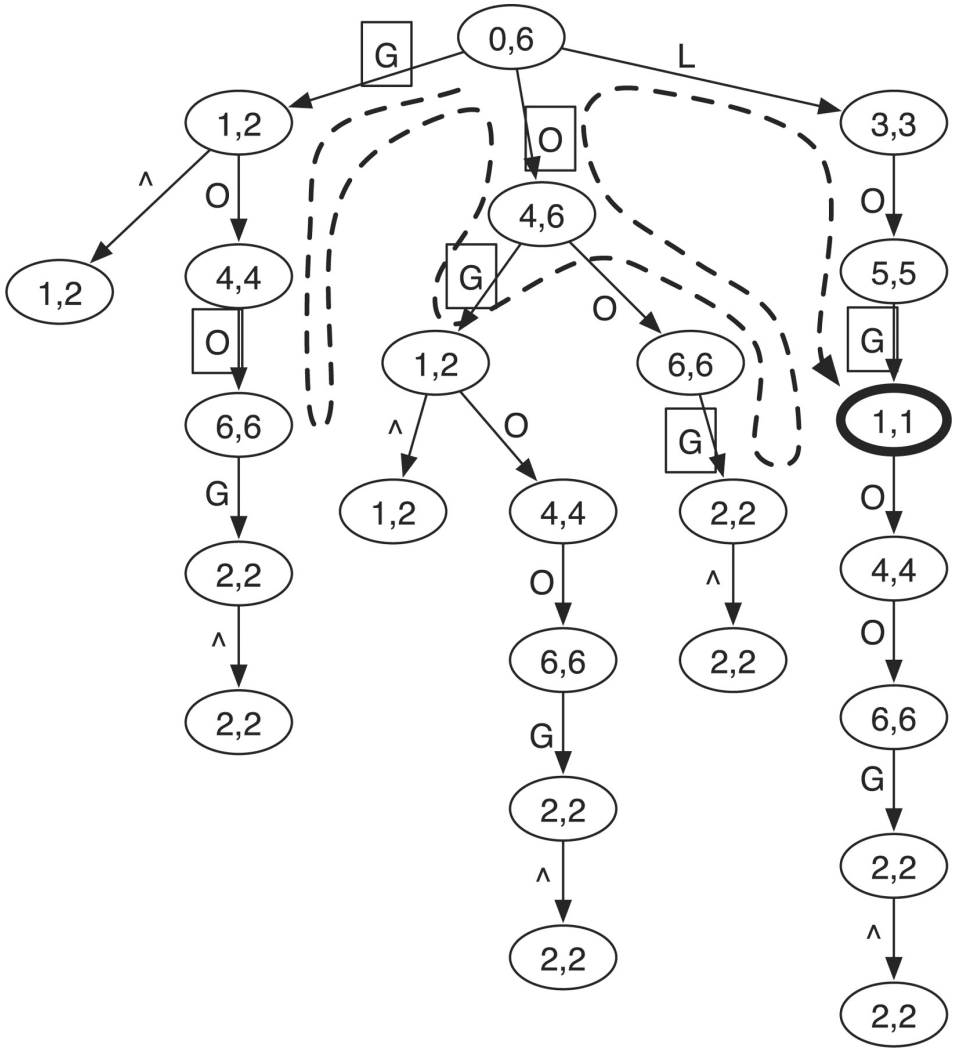


Inexact matching

Consider now the role of the D matrix in the DFS shown in the figure

- The recursive call looks like this: $\texttt{InexRecur}(W, 1, 0, 1, 2)$
  Because in the previous iteration:
  - $i \leftarrow i - 1 = 1$
  - $z \leftarrow z - 1 = 0$
  - $k \leftarrow C('G') + O('G', k-1) + 1 = 1$
  - $\ell \leftarrow C('G') + O('G', \ell) = 2$

- When we get to line 4, $i = 1$ and $z = 0$. Recalling that we calculated $D(1) = 1$, we have that $z < D(i)$, and we return without having examined the subtree emanating from 'G'
- Similarly, we avoid descending into the 'O' subtree
- Only for the 'L' subtree we will continue until we find the match



References I



Li H. & Durbin R. (2009)
Fast and accurate short read alignment with Burrows-Wheeler transform
Bioinformatics 25:1754-60